

C-Trace: An Open-Source RISC-V Trace Encoder and its Ecosystem

Alexander Weiss¹, Albert Schulz¹, Michael Faulwasser², Eyck Jentzsch³,
Simon Wegener⁴, Dominik Jürgens⁵, and Darshak Sheladiya⁶

¹Accemic Technologies GmbH, ²Fraunhofer Institute for Photonic Microsystems (IPMS), ³MINRES Technologies GmbH,
⁴AbsInt Angewandte Informatik GmbH, ⁵tensor embedded GmbH, ⁶SYSGO GmbH

Abstract

Embedded tracing is essential for validating reliability, optimizing performance, and debugging complex embedded software. Despite rapid innovation in the RISC-V ecosystem, open and interoperable trace solutions have remained limited. C-Trace, developed in the context of the European TRISTAN project¹, addresses this gap with an open-source trace encoder and an extensible ecosystem approach. C-Trace introduces a modular trace-encoder architecture designed for efficient, continuous “live” observation. Beyond standard program-flow tracing, it supports hardware-assisted instrumentation that can automatically emit trace messages on access to selected control/status registers (CSRs) or on configurable watchpoints. This enables trace streams that carry richer runtime context (e.g., program counter, timestamps, direct data, and selected performance counters) and can support application use cases such as worst-case execution time (WCET) estimation, timing optimization, test-case prioritization, and integration-level coverage measurement. In addition to off-chip export, C-Trace can forward trace-triggered events internally to an on-chip CPU, enabling watchdog, runtime verification, or control-flow integrity (CFI) checking functionality. Finally, C-Trace is provided under a dual-licensing model (CERN-OHL-S and a non-copyleft commercial option) to balance open collaboration with industrial IP needs.

Introduction

Trace-based dynamic analysis is a cornerstone of robust embedded development workflows, as it enables diagnosis of rare faults, timing regressions, integration issues, and security violations that often remain invisible to logs or intrusive instrumentation. At the same time, embedded systems face rising complexity and stronger requirements in safety-critical and security-aware domains (e.g., automotive, avionics, railway), which increases the need for deterministic, low-intrusion observation mechanisms.

While the RISC-V ecosystem is advancing quickly, many trace solutions remain either proprietary, platform-specific, or difficult to integrate into open toolchains. C-Trace is designed to be an open, extensible trace encoder that fits into a broader interoperability scheme, supporting both practical debugging and continuous “live-analysis” scenarios.

Contributions

- A modular trace-encoder architecture for RISC-V platforms, designed for continuous observation and integration with ecosystem tooling.
- A hardware-supported instrumentation (HSI) interface with both CSR-triggered and watchpoint-driven actions

that enrich the trace stream with timestamps, direct data, and specialized counters.

- A concept for internal routing of trace-triggered events to an on-chip CPU to enable runtime verification functions like a watchdog or control-flow integrity (CFI) checking.
- An open governance and dual-licensing approach² suitable for research and industry collaboration via CERN-OHL-S and a non-copyleft commercial license option.

System Overview

Key engineering challenges addressed by C-Trace’s architecture include balancing trace richness with bandwidth constraints through selective triggers and configurable formatting; and enabling both off-chip observability and low-latency on-chip reactions for safety and security use cases. To achieve this, the architecture separates concerns into pipelined stages that can evolve independently. At a high level, C-Trace provides:

- An interface to RISC-V processors via the standardized Trace Ingress Port³ (TIP)
- A configurable trace-encoder core driven by control/status registers (CSRs).

¹ The TRISTAN project, nr. 101095947 is supported by Chips Joint Undertaking (CHIPS-JU) and its members Austria, Belgium, Bulgaria, Croatia, Cyprus, Czechia, Germany, Denmark, Estonia, Greece, Spain, Finland, France, Hungary, Ireland, Israel, Iceland, Italy, Lithuania, Luxembourg, Latvia, Malta, Netherlands, Norway, Poland, Portugal, Romania, Sweden, Slovenia, Slovakia, Turkey and including top-up funding by the German Federal Ministry of Education and Research.

² See <https://accemic.com/c-trace/> for details.

³ <https://github.com/riscv-non-isa/tg-nexus-trace/blob/main/docs/RISC-V-N-Trace.adoc#trace-ingress-port>

- An extended preprocessing stage that adds instrumentation-driven trace events and counters.
- Formatting stages that emit trace in ecosystem-compatible forms, including IEEE-ISTO 5001-2003 Nexus-compatible flows and configurable message widths.

Register and Configuration Management (SystemRDL as Single Source of Truth)

C-Trace uses SystemRDL-based register management to keep the hardware/software interface consistent and maintainable: from a single SystemRDL specification, the project generates RTL, software headers, and documentation, organized with a clean addrmap/regfile structure. This reduces integration friction, supports automation, and minimizes configuration mismatches between firmware and hardware.

Instrumentation Beyond Basic Program Trace

A key aspect of C-Trace is its “extended preprocessing stage,” which augments the program trace with event-driven instrumentation and measurement. It provides a compact, configurable method to encode “semantic” runtime events into the trace stream via the HSI interface.

The submodule ACT-CAP provides a CSR Access Protocol that enables controlled access to selected trace encoder control registers and supports emitting trace messages that include PC, timestamp, and direct data. It also manages special counters such as wall clock counter, stall counter, cache miss counter, and read/write counts to configurable memory regions. These features allow trace data to be used for performance diagnostics and for analysis tasks that require time and event correlation.

Additionally, the submodule ACT-ST enables “Smart Trace” actions triggered by watchpoints. This mechanism can be used to limit trace recording to relevant execution windows only (e.g., around critical functions, state transitions, or suspected fault regions), which is particularly valuable under tight trace bandwidth constraints.

The design includes performance counters with programmable instruction/data address ranges, counting events such as data reads, data writes, or instruction fetches exceeding configurable CPU-cycle thresholds. These counters support continuous performance monitoring and help locate timing anomalies or regressions.

Beyond off-chip export, C-Trace can provide uncompressed trace data via AXI4-Stream and enable on-chip use cases such as watchdog, runtime verification, CFI checking, and JIT optimization. This internal routing is important when immediate reaction is required (e.g., fail-

safe transitions) or when off-chip bandwidth/availability is limited.

Formatting and Output: Ecosystem Compatibility and Configurable Encodings

C-Trace includes a formatting stage that can adapt output encodings, including an extended MESO/MDO formatter with variable MDO widths. By supporting flexible output widths and structured formatting, C-Trace targets practical integration with existing trace transport and decoder infrastructures, including Nexus-oriented ecosystems.

Application Use Cases Enabled by C-Trace

C-Trace is positioned for continuous live observation and supports application-driven outcomes including timing analysis and optimization, test case prioritization, CFI checking, and code coverage measurement at integration-test level. A distinctive aspect is that these use cases can be served by hardware-driven trace events (CSR/watchpoint triggers) and optionally by internal on-chip consumers, enabling both offline analysis and in-system runtime enforcement/monitoring.

The trace unit has been integrated prototypically into two RISC-V cores (EMSA5 of Fraunhofer IPMS⁴ and TGC of MINRES Technologies⁵), while work to integrate it into CVA6 is under way, for which a TIP interface has been developed by SYSGO⁶. There is already a growing ecosystem around C-Trace. First tools to utilize the traces include AbsInt’s TimeWeaver⁷, a hybrid WCET estimation tool for safety-critical embedded software, and Tensor’s Timealyzer⁸, a streaming trace-analytics front-end that turns continuous trace data into decision-ready metrics. Timealyzer correlates hyperperiod-based timing patterns across tasks/event chains and links rare outliers back to instruction-level traces and build artefacts for targeted schedule and code optimization.

Conclusion

C-Trace is an open-source, modular trace encoder for RISC-V which goes beyond basic program tracing. It uses hardware-supported instrumentation (using CSR and watchpoint triggers), structured formatting with configurable widths and optional internal on-chip event consumption. These capabilities enable a variety of continuous analysis use cases, ranging from performance and timing diagnostics to runtime verification and CFI checking, while supporting an ecosystem-oriented approach and a dual-licensing model that is suitable for both research and industry.

⁴ <https://doi.org/10.24406/publica-4544>

⁵ <https://www.minres.com/the-good-folk-series/>

⁶ <https://github.com/openhwgroup/cva6/commit/f314dcb136ed373db8332e30a897fa06db4aae43>

⁷ <https://www.absint.com/timeweaver/>

⁸ <https://www.tensor-embedded-gmbh.com/produkte>